

The Implementation of System Fun_ω

Programming Language Implementation and Formalisation (Spring 2026)

During the lectures, we have introduced the Fun programming language. The Fun language that supports higher order functions, type inference and a couple of built-in algebraic types. The following assignment text concerns itself with extending the Fun programming language with user definable operators, algebraic data types and pattern matching.

The assignment is accompanied by a zip folder. You are expected to implement your solution in the sub-folder `implementation`, and you are expected to change the driver such that the command `"cabal run"` will launch an interactive read eval print loop (REPL). You can add several test suites, but the command `"cabal test all"` is expected to run your tests.

This text is not separated into two parts. But for the possibility, it is suggested that you implement the parser and start with the interpreter. For the home-exam, it will be the full project.

WHAT TO HAND IN

The hand-in for Home Exam 2 consists of three parts, a parser, a type checker, and an interpreter. We expect you to hand in the zip folder containing your solution to all three parts, and a single pdf, containing a 2-3 page assessment of your work.

As an additional service, this year, we offer an optional assignment, in this assignment, you can hand in the parser and interpreter to get feedback on your work.

You can choose to hand in this assignment via Devilry, and will also receive feedback via Devilry. The assignment is called Possiblix 2.



PART 1 - SYNTAX

The goal of this assignment is to gain experience in implementing a programming language from its formalisation. You will be implementing a parser and interpreter for the Fun_ω programming language. You can find a BNF of the syntax in Figure 1. The syntax of Fun_ω is not context free, so remainder of this section will provide a disambiguation.

Fun_ω supports various names, the meaning of *well-formed* in Figure 1 is as follows

- The well-formed variable names are lowercase letters followed by alphanumeric characters. However, keywords such as `case` and `of`, are not valid variable names.
- The well-formed constructor names are uppercase letters followed by alphanumeric characters.
- A well-formed operator name, is any combination of non-alphanumeric characters that are also not space or control characters. The names `"."`, `"="` and `":"` are not allowed.

$n \in \mathbb{N}$	(Natural numbers).
$x \in \mathbf{Name}$	(Well-formed variable names).
$c \in \mathbf{Name}$	(Well-formed constructor names).
$\oplus \in \mathbf{Name}$	(Well-formed operator names).
$d \in \mathbf{Name}$	(Well-formed datatype names).
$\mathbb{F} ::= \text{infixr} \mid \text{infixl} \mid \text{infix}$	(Fixity Declaration).
$\mathbb{P} ::= \text{data } d \ x_i = (c \ \tau_i)_j . \mathbb{P}$	(Data type definition).
$ \mathbb{F} \ n \ x_0 \oplus x_1 = t . \mathbb{P}$	(Binary operator definition).
$ x : \tau = t . \mathbb{P}$	(Variable definition).
$ \varepsilon$	(The empty program).
$\tau ::= d \ x_i \mid \tau_0 \rightarrow \tau_1 \mid x$	(Types).
$t ::= x$	(Variable).
$ c \ t_i$	(Algebraic Term).
$ t_0 \ t_1$	(Term Application).
$ t_0 \oplus t_1$	(Operator Application).
$ \text{fun } x \rightarrow t$	(Function term).
$ \text{case } t_0 \text{ of } (; p \rightarrow t)_i$	(Case term).
$p ::= x \mid c \ p_i$	(Patterns).
$v ::= c \ v_i$	(Algebraic Values).
$ \lambda x . t$	(Function Values).

Figure 1. The syntax of Fun_ω . The subscript notation denotes finite indexed sequences, ranging over the naturals, and starting from 0. For instance, a datatype declaration might look like `data List a = | Nil | Cons a (List a) .`

```

data Bool = | True | False .

infixr 4 b0 & b1 =
  case b0 of
  ; True  -> b1
  ; False -> False .

infixl 3 b0 | b1 =
  case b0 of
  ; True  -> True
  ; False -> b1 .

not : Bool -> Bool =
  fun b ->
    case b of
    ; True -> False
    ; False -> True .

xor : Bool -> Bool -> Bool =
  fun b0 ->
  fun b1 -> b0 | b1 & not (b0 & b1) .

```

Figure 2. An example program, written in Fun_ω

- A data type name is well formed, if it is a well-formed constructor name.

A program $\mathbb{P}$ written in Fun_ω is a sequence of mutually recursive punctuation-terminated definitions. There are three kinds of definition. A datatype definition consists of a name, a (possibly empty) list of parameters, and a sequence of constructor definitions. An operator pattern consists of a fixity $\mathbb{F}$, a precedence level, two argument variables and a body term. Finally, a definition can define a (typed) variable. For instance, the program in Figure 2 contains a datatype definition, two operator definitions and two variable definitions:

The last line is to be parsed as $(b0 \mid b1) \ \& \ (\text{not } (b0 \ \& \ b1))$, because the binary operator $\mid$ has been declared to have lower precedence (binding tighter) than the operator $\&$.

To parse a program, you must therefore iterate through the program twice (first in order to collect all of the fixity definitions, and secondly to parse the program). For this purpose, the program provides a datatype `OpTable`. As hinted by the type of `OpTable` it is an error if two operators on the same precedence level have different fixity.

TYPE SYSTEM

Given a typing environment Γ , the typing judgement for Fun_ω can be found in Figure 3. There are six rules: The `Variable` rule, says that a variable has the type τ if Γ maps x to τ . The `Constructor` rule says that a constructor c applied to terms t_i has the type $d\tau_i$, if $\mathbb{P}$ contains a data type definition in which one of the constructors is called c , and if we can infer that each term t_i has the type τ_i (pairwise).

$$\boxed{\Gamma \mathbb{P} \vdash t : \tau}$$

$$\begin{array}{ll} \text{Variable:} : \frac{}{\Gamma[x \rightarrow \tau] \mathbb{P} \vdash x : \tau} & \text{Constructor:} : \frac{\Gamma \mathbb{P} \vdash t_i : \tau_i}{\Gamma \mathbb{P}[\text{data } d \ x_i = c \ \tau_i] \vdash c \ t_i : d \ \tau_i} \\ \text{Abstraction:} : \frac{\Gamma[x \rightarrow \tau_x] \mathbb{P} \vdash t : \tau}{\Gamma \mathbb{P} \vdash \text{fun } x \rightarrow t : \tau_x \rightarrow \tau} & \text{Application:} : \frac{\Gamma \mathbb{P} \vdash t_0 : \tau_1 \rightarrow \tau \quad \Gamma \mathbb{P} \vdash t_1 : \tau_1}{\Gamma \mathbb{P} \vdash t_0 \ t_1 : \tau} \\ \text{Cases:} : \frac{\Gamma \mathbb{P} \vdash t : \tau' \quad \Gamma[p_i \rightarrow \tau'] \mathbb{P} \vdash t_i : \tau}{\Gamma \mathbb{P} \vdash \text{case } t \text{ of } p_i \rightarrow t_i : \tau} & \text{Operator:} : \frac{\Gamma[x_0 \rightarrow \tau_0, x_1 \rightarrow \tau_1] \mathbb{P} \vdash t : \tau}{\Gamma \mathbb{P}[x_0 \oplus x_1 = t] \vdash t_0 \oplus t_1 : \tau} \end{array}$$

Figure 3. The typing judgement (with Γ , in the program $\mathbb{P}$, the term t has type τ).

$$\boxed{\mathbb{P} \vdash t \downarrow v}$$

$$\begin{array}{ll} \text{Variable}_{\downarrow} : \frac{\mathbb{P} \vdash t \downarrow v}{\mathbb{P}[x \rightarrow t] \vdash x \downarrow v} & \text{Constructor}_{\downarrow} : \frac{\mathbb{P} \vdash t_i \downarrow v_i}{\mathbb{P} \vdash c \ t_i \downarrow c \ v_i} \\ \text{Abstraction}_{\downarrow} : \frac{}{\mathbb{P} \vdash \text{fun } x \rightarrow t \downarrow \lambda x. t} & \\ \text{Application}_{\downarrow} : \frac{\mathbb{P} \vdash t_0 \downarrow \lambda x. t \quad \mathbb{P} \vdash t_1 \downarrow v_1 \quad \mathbb{P} \vdash t[v_1/x] \downarrow v}{\mathbb{P} \vdash t_0 \ t_1 \downarrow v} & \\ \text{FirstCase}_{\downarrow} : \frac{\mathbb{P} \vdash t \downarrow v_0 \quad \text{mgu}(p_0, v_0) = \sigma \quad \mathbb{P} \vdash \sigma(t_0) \downarrow v}{\mathbb{P} \vdash \text{case } t \text{ of } p_i \rightarrow t_i \downarrow v} & \\ \text{NextCase}_{\downarrow} : \frac{\mathbb{P} \vdash t \downarrow v_0 \quad \text{mgu}(p_0, v_0) = \mathbf{fail} \quad \mathbb{P} \vdash \text{case } t \text{ of } p_i \rightarrow t_i \downarrow v}{\mathbb{P} \vdash \text{case } t \text{ of } ; p_0 \rightarrow t_0 ; p_i \rightarrow t_i \downarrow v} & \end{array}$$

Figure 4. The evaluation judgement for Fun_{ω}

INTERPRETATION

In Figure 4, you will find the operational semantics of Fun_{ω} . The $\text{Variable}_{\downarrow}$ holds if we can look up the variable in our program. The $\text{Constructor}_{\downarrow}$ rule holds if we can find a derivation for each component that the constructor is applied to.

The $\text{FirstCase}_{\downarrow}$ rule says that if the selector of a case expression evaluates to some value v_0 , and the first pattern p_0 unifies with this value, then we proceed by evaluating the body of the first case. On the contrary, the $\text{NextCase}_{\downarrow}$ rule says that we must throw away the first case if it fails to unify.

FOR THE REPORT

The rules in Figure 4 does not provide a rule for binary operators. Suggest a rule that describes the intended operational semantics, and/or disambiguate binary operators into something which is already supported in Fun_{ω} .